

Density-Aware Geodesic Distance Implementation Design Memo

Density-Aware Geodesic Distance Implementation Design Memo

1. Design Goals
2. Existing Assets To Preserve
3. Core Data Objects
 - Density Estimate Object
 - Edge Density Table
 - Density-Aware Graph Object
4. Internal Helper Functions
 - Density Estimation
 - Density Transforms
 - Edge Density Summaries
 - Graph Reweighting
 - Fermat / PWSPD Comparator
 - Density-Aware Graph Construction Wrapper
5. Distance Families To Expose Internally
6. Diagnostic Functions
 - Density Diagnostics
 - Edge Diagnostics
 - Path Diagnostics
 - Distance Comparison Diagnostics
7. Defaults For Internal Benchmarks
8. Noisy-Circle / Tube Integration
9. Compositional and Microbiome-Like Data
10. Future Public API
11. Guardrails
12. Testing Strategy
13. Report Generation Convention
14. Implementation Phases
 - Phase 1: Internal Graph Reweighting
 - Phase 2: Robust Oracle Refinement
 - Phase 3: Topology Before Weighting
 - Phase 4: Compositional Synthetic Data
 - Phase 5: Public API Decision
15. Open Design Questions
16. Summary Recommendation

Density-Aware Geodesic Distance Implementation Design Memo

Date: 2026-05-04

This memo is Deliverable 3 for the density-aware geodesic-distance research track. Deliverable 1 mapped the literature; Deliverable 2 fixed the formal definitions. This memo proposes an implementation path for `gflow`, focused on internal development helpers, diagnostics, benchmark integration, and a future public API. It is intentionally conservative: density-aware distance should first live in the development workspace until the benchmark reports show stable behavior.

1. Design Goals

The implementation should support three distinct tasks:

1. define density-aware truth metrics for synthetic benchmarks;
2. estimate density-aware graph geodesics from finite data;
3. compare shortest-path, Fermat/PWSPD, diffusion, PHATE, and oracle distances without mixing their meanings.

The immediate implementation target is not a public CRAN-facing API. The next step should be a small internal toolkit under `dev/geodesic-distance-estimation/R/`, designed to generate benchmark reports with enough diagnostics to decide whether any public API is justified.

The current helper style in `dev/geodesic-distance-estimation/R/graph_construction.R` is list-based, dot-delimited, and report-oriented. New development helpers should follow that style.

2. Existing Assets To Preserve

The current code already provides the core graph and oracle primitives:

- `edge.table()`, `graph.from.edges()`, `union.graphs()`, `igraph.from.graph()`, and `graph.distances()`;
- `iknn.graph()`, `mknn.graph()`, `adaptive.threshold.graph()`, `prune.redundant.edges()`, and `cycle.repair.graph()`;
- `geodesic.metrics()`, `graph.summary.metrics()`, and `score.graph.metrics()`;
- `tube.coords()`, `tube.density()`, `latent.tube.oracle()`, and `latent.tube.oracle.distances()`.

These helpers should not be replaced. Density-aware implementation should compose with them. The main extension is to add a density-estimation layer, an edge-density summary layer, and a graph-reweighting layer.

3. Core Data Objects

Density Estimate Object

An internal density estimate should be a plain list with a stable structure:

```
list(
  values = numeric(n),
  transformed = numeric(n),
  method = "knn_radius",
  space = "input",
  dimension = d.hat,
  params = list(...),
  diagnostics = data.frame(...)
)
```

Definitions:

$\hat{\rho}_i$ = raw estimated density at vertex i ,

$$\rho_i^* = T(\hat{\rho}_i)$$

where T applies floors, caps, and robust transforms.

The object should record both raw and transformed density. Downstream graph weighting should use transformed, while reports should show both.

Edge Density Table

A density-aware edge table should extend the current `edge.table()` output:

```
data.frame(
  from = integer(),
  to = integer(),
  base_weight = numeric(),
  rho_from = numeric(),
  rho_to = numeric(),
  rho_edge = numeric(),
  penalty = numeric(),
  weight = numeric(),
  floor_hit = logical(),
  cap_hit = logical(),
  summary = character(),
  edge_kind = character()
)
```

The density-aware edge weight is

$$w_{ij} = \ell_{ij} \psi(\rho_{ij}^*; \alpha),$$

where the standard penalty is

$$\psi(\rho; \alpha) = \rho^{-\alpha}.$$

Equivalently,

$$w_{ij} = \frac{\ell_{ij}}{(\rho_{ij}^*)^\alpha}.$$

The edge table is the right place to preserve attribution information: whether the edge came from MST, CMST completion, iKNN, mKNN, component repair, oracle grid, or sample attachment.

Density-Aware Graph Object

The graph object should remain compatible with existing helpers:

```
list(
  adj_list = list(...),
  weight_list = list(...),
  base_weight_list = list(...),
  edge_density = data.frame(...),
  density = density.object,
  density_weighting = list(...),
  diagnostics = list(...)
)
```

Existing code can continue to consume `adj_list` and `weight_list`. New code can inspect `base_weight_list`, `edge_density`, and `diagnostics`.

4. Internal Helper Functions

Density Estimation

Proposed internal helpers:

```
estimate.density.knn <- function(X, k = 10L, dimension = NULL,
                                normalize = TRUE, ...)

estimate.density.kernel <- function(X, bandwidth = NULL, dimension =
  NULL,
                                adaptive = FALSE, ...)

estimate.density.graph <- function(graph, mode = c("degree",
  "weighted_degree",
  "volume_growth"),
                                ...,
                                "stationary",
                                ...)

estimate.density.random.walk <- function(graph, t, start = NULL,
                                summary = c("entropy",
  "return_mass",
  "effective_support"),
                                ...)
```

The kNN-radius estimator should be the first implementation because it is simple, fast, and directly tied to Fermat/PWSPD theory:

$$\hat{\rho}_i = \frac{k}{n c_{\hat{d}} r_k(i)^{\hat{d}}},$$

where $r_k(i)$ is the distance to the k th nearest neighbor and $c_{\hat{d}}$ is a dimension-dependent volume constant. For graph weighting, the constant can be dropped if only relative density matters.

Density Transforms

Proposed helper:

```
transform.density <- function(rho, floor = NULL, cap = NULL,
                             normalize = c("median", "mean", "max",
                                           "none"),
                             penalty.cap = NULL,
                             robust = c("none", "log", "rank"),
                             alpha = 0.5)
```

Default internal behavior should normalize transformed density so that the median is one:

$$\text{median}(\rho_i^*) = 1.$$

This makes density-weighted edge lengths comparable to base edge lengths and reduces accidental global rescaling across seeds.

The transform object should report:

- floor value and number of vertices at the floor;
- cap value and number of vertices at the cap;
- penalty cap and number of vertices controlled by it;
- raw and transformed density quantiles;
- median penalty $(\rho^*)^{-\alpha}$.

Edge Density Summaries

Proposed helper:

```
edge.density.summary <- function(edges, density, X = NULL,
                                 summary = c("geometric", "harmonic",
                                             "arithmetic", "minimum",
                                             "midpoint",
                                             "edge_neighborhood"),
                                 ...)
```

Recommended internal default: "geometric" for ordinary local edges and "minimum" or "edge_neighborhood" for bridge diagnostics.

Formulas:

$$\phi_{ij}^{\text{geom}} = \sqrt{\rho_i^* \rho_j^*},$$

$$\phi_{ij}^{\text{harm}} = \frac{2}{(\rho_i^*)^{-1} + (\rho_j^*)^{-1}},$$

$$\phi_{ij}^{\text{min}} = \min\{\rho_i^*, \rho_j^*\}.$$

Midpoint density is useful only when the density estimator can evaluate new locations:

$$\phi_{ij}^{\text{mid}} = \hat{\rho}^*\left(\frac{x_i + x_j}{2}\right).$$

Graph Reweighting

Proposed helper:

```
density.weight.graph <- function(graph, density, X = NULL,
                                alpha = 0.5,
                                edge.summary = "geometric",
                                preserve.base.weights = TRUE,
                                penalty.cap = NULL,
                                edge.kind = NULL)
```

This should never change graph topology. It only changes edge weights.

The returned graph should satisfy:

$$w_{ij}^{(\alpha=0)} = \ell_{ij}.$$

That identity is an important test.

Fermat / PWSPD Comparator

Proposed helper:

```
fermat.weight.graph <- function(graph, p = 2,
                                rooted = TRUE,
                                preserve.base.weights = TRUE)
```

For each edge,

$$w_{ij}^{\text{Fermat}} = \ell_{ij}^p.$$

Shortest paths on these weights produce the unrooted Fermat convention. If the rooted convention is requested, distances should be transformed after shortest paths:

$$\ell_p(i, j) = d_{X,p}(i, j)^{1/p}.$$

The helper should record:

$$\alpha_{\text{implied}} = \frac{p-1}{\hat{d}}$$

when an intrinsic dimension estimate is provided.

Density-Aware Graph Construction Wrapper

Proposed internal wrapper:

```
create.density.aware.graph <- function(X,
                                     graph.method = c("iknn",
                                     "mknn", "cmst",
                                     "mst",
                                     "given"),
                                     graph = NULL,
                                     density.method = "knn_radius",
                                     density.params = list(),
                                     density.transform = list(),
                                     alpha = 0.5,
                                     edge.summary = "geometric",
                                     diagnostics = TRUE,
                                     ...)
```

This wrapper is for experiments, not public API. It should:

1. construct or accept a base graph;
2. estimate density in the same coordinate space used for graph construction;
3. transform density;
4. summarize density along edges;
5. reweight edges;
6. attach diagnostics.

5. Distance Families To Expose Internally

Benchmark reports should compare named distance families:

Name	Construction	Output type
euclidean	pairwise Euclidean or preprocessed distance	metric
base_graph_geodesic	shortest paths on base graph weights	graph metric
density_weighted_graph	shortest paths on $w_{ij} = \ell_{ij} / \phi_{ij}^\alpha$	graph metric
fermat_unrooted	shortest paths on ℓ_{ij}^p	graph metric
fermat_rooted	fermat_unrooted raised to $1/p$	metric
diffusion	distance between rows of P_a^t	diffusion/pseudometric
phate_potential	distance between transformed diffusion potentials	potential/pseudometric
oracle_density_weighted	known-density continuous or grid truth	oracle metric

The report layer should never collapse these into one generic “distance” without preserving the family name and parameterization.

6. Diagnostic Functions

Density Diagnostics

Proposed helper:

```
density.diagnostics <- function(density)
```

Required output:

- raw density quantiles;
- transformed density quantiles;
- penalty quantiles for the active α ;
- floor and cap hit rates;
- correlation with kNN radius, degree, and base edge length if available.

Edge Diagnostics

Proposed helper:

```
density.edge.diagnostics <- function(weighted.graph)
```

Required output:

- base edge length versus density-weighted edge length;
- ρ_i , ρ_j , and ρ_{ij} summaries;
- penalty distribution;
- top high-penalty edges;
- top edge inflation ratios;
- floor/cap edges;
- bridge-like edges ranked by length and density penalty.

Edge inflation ratio:

$$I_{ij} = \frac{w_{ij}}{\ell_{ij}} = \phi_{ij}^{-\alpha}.$$

Path Diagnostics

Proposed helper:

```
density.path.diagnostics <- function(weighted.graph, pairs,
                                     base.graph = NULL,
                                     truth.dist = NULL)
```

For a selected path π , report:

$$L_{\text{base}}(\pi) = \sum_{(i,j) \in \pi} \ell_{ij},$$

$$L_{\text{density}}(\pi) = \sum_{(i,j) \in \pi} w_{ij},$$

and path-level density summaries:

$$\min_{(i,j) \in \pi} \phi_{ij}, \quad \text{median}_{(i,j) \in \pi} \phi_{ij}, \quad \max_{(i,j) \in \pi} I_{ij}.$$

Distance Comparison Diagnostics

Existing `geodesic.metrics()` should be extended or wrapped to compare several distance matrices at once:

```
compare.distance.matrices <- function(distances, truth = NULL,
                                     labels = names(distances),
                                     k.neighborhood = 10L)
```

Required comparisons:

- Pearson and Spearman correlations;
- optimal global scale factor;
- median and upper-quantile relative error;
- nearest-neighbor overlap;
- finite-pair fraction;
- pairwise matrix correlations among Euclidean, base graph, density-weighted, Fermat, diffusion, PHATE, and oracle distances.

7. Defaults For Internal Benchmarks

Initial benchmark defaults should be conservative:

```
alpha_grid <- c(0, 0.25, 0.5, 0.75, 1)
diagnostic_alpha <- 0.5
density_floor_quantile <- 0.02
density_cap_quantile <- 0.98
edge_summary <- "geometric"
bridge_edge_summary <- "minimum"
```

Recommended interpretation:

- $\alpha = 0$ is the base graph control.
- $\alpha = 0.25$ and $\alpha = 0.5$ are moderate density-aware regimes.
- $\alpha = 0.75$ and $\alpha = 1$ are stress tests for tail domination.

No public function should default to aggressive density weighting. If a future public API enables density-aware weighting, the default should either be `alpha = 0` or require an explicit opt-in such as `density.weighting = list(alpha = 0.5, ...)`.

8. Noisy-Circle / Tube Integration

The existing `latent.tube.oracle()` is already the right place to continue oracle development. It should be extended, not rewritten.

Proposed refinements:

1. return edge-level density and penalty columns in `oracle$edges`;
2. allow `density.cap` and `penalty.cap`;
3. allow alternative edge density summaries beyond midpoint u ;
4. add grid-resolution diagnostics;
5. add sample-attachment diagnostics;
6. compare oracle distances across α , floor, cap, and attachment rules.

The existing oracle weight

$$w_{ab} = \frac{\|z_a - z_b\|}{\rho(m_{ab})^\alpha}$$

should remain the baseline.

9. Compositional and Microbiome-Like Data

For microbiome and metagenomic data, density estimation must be tied to preprocessing. The design should support density spaces:

```
density.space <- c("raw_counts", "relative_abundance", "clr",
                  "aitchison",
                  "pca_clr", "graph", "model_latent")
```

Initial recommendation:

1. do not estimate density in raw counts unless modeling library size explicitly;
2. prefer CLR/Aitchison or PCA after compositional preprocessing for synthetic microbiome-like benchmarks;
3. always report whether density correlates with library size, zero fraction, batch, or sequencing depth;
4. treat density weighting as biologically meaningful only if those nuisance correlations are controlled.

Required microbiome-specific diagnostics:

- library size versus density;
- zero fraction versus density;
- batch or cohort versus density;
- rare-taxon filtering sensitivity;
- pseudocount sensitivity for CLR;
- comparison to Aitchison, robust Aitchison, Bray-Curtis, UniFrac when available.

10. Future Public API

Public API should wait until benchmark evidence is stronger. If promoted, the API should separate density estimation, graph weighting, and distance computation.

Possible public functions:

```
estimate.density <- function(x, method = "knn_radius", space = NULL,
  ...)
```

```
density.weight.graph <- function(graph, density, alpha,
  edge.summary = "geometric", ...)
```

```
geodesic.distance <- function(graph, metric = c("base",
  "density_weighted",
  "fermat"),
  ...)
```

```
create.density.aware.graph <- function(x, graph.method,
  density.method,
  alpha, ...)
```

Possible integration hooks:

```
create.iknn.graphs(..., density.weighting = NULL)
create.cmst.graph(..., density.weighting = NULL)
phate.core(..., density.aware = NULL)
```

Recommended public design principle: density estimation should be reusable. Users should be able to inspect, plot, and diagnose density before it changes a distance matrix.

11. Guardrails

Every density-aware run should fail loudly or warn when:

1. more than a chosen fraction of vertices hit the density floor;
2. more than a chosen fraction of edges hit the penalty cap;
3. density is strongly correlated with a known nuisance variable;
4. the density-weighted graph has extreme edge inflation;
5. density-weighted distances are dominated by a small number of tail edges;
6. graph topology contains unsupported shortcuts that density summaries cannot detect;
7. diffusion or PHATE outputs are being labeled as geodesic distances.

Suggested warning thresholds for internal reports:

```
max_floor_fraction <- 0.10
max_penalty_cap_fraction <- 0.10
max_edge_inflation_q99 <- 20
max_library_size_density_cor <- 0.5
```

These thresholds are not final defaults. They are report triggers.

12. Testing Strategy

Internal tests should begin as small deterministic checks in the development workspace before package tests are added.

Minimum unit-style checks:

1. $\alpha = 0$ reproduces base graph distances exactly;
2. constant density reproduces base graph distances up to a global scale;
3. disconnected graphs produce infinite between-component distances;
4. zero or negative transformed densities are rejected;
5. edge summaries are symmetric for undirected graphs;
6. Fermat rooted and unrooted conventions agree by $d_{X,p}^{1/p}$;
7. floor and cap diagnostics count affected vertices and edges correctly;
8. oracle distances converge as grid resolution increases in a small case.

Benchmark checks:

1. noisy circle/tube with $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$;
2. uneven line and circle with angular density gaps;
3. nearby non-touching arcs;
4. compositional cyclic gradient;
5. outliers and batch-like nuisance variation.

13. Report Generation Convention

From this deliverable onward, every memo/report in this density-aware track should have three sibling artifacts when practical:

```
*.md
*.html
*.pdf
```

HTML should be generated with Pandoc and MathJax-compatible math parsing:

```
pandoc input.md \
  --standalone \
  --from=markdown+tex_math_dollars+tex_math_single_backslash \
  --to=html5 \
  --mathjax \
  --toc \
  --toc-depth=3 \
  -o output.html
```

PDF should be generated from the HTML through headless Chrome and then rewritten through Ghostscript. The Ghostscript pass is required because HeadlessChrome/Skia PDFs can include font and tagged-PDF structures that render correctly on macOS but fail on some AirPrint/PostScript printer pipelines.

Use the shared renderer:

```
dev/geodesic-distance-estimation/tools/render_report.sh input.md
```

Equivalent explicit commands:

```
"/Applications/Google Chrome.app/Contents/MacOS/Google Chrome" \
--headless \
--disable-gpu \
--no-sandbox \
--run-all-compositor-stages-before-draw \
--virtual-time-budget=10000 \
--print-to-pdf=output.pdf \
"file:///absolute/path/output.html"
```

```
gs \
-dSAFER \
-dBATCH \
-dNOPAUSE \
-sDEVICE=pdfwrite \
-dCompatibilityLevel=1.4 \
-dWriteObjStms=false \
-dWriteXRefStm=false \
-dPDFSETTINGS=/prepress \
-dEmbedAllFonts=true \
-dSubsetFonts=true \
-dCompressFonts=true \
-dDetectDuplicateImages=true \
-dAutoRotatePages=/None \
-sOutputFile=output.print-safe.pdf \
output.pdf
```

```
mv output.print-safe.pdf output.pdf
```

The final report check should include:

1. no raw TeX display delimiters in extracted PDF text;
2. a visual spot-check of pages containing multi-line formulas;
3. a page count and file-size check;
4. confirmation that the canonical PDF has passed through Ghostscript;
5. `git diff --check` on Markdown and HTML sources.

14. Implementation Phases

Phase 1: Internal Graph Reweighting

Add density objects, density transforms, edge summaries, graph reweighting, and diagnostics in `dev/geodesic-distance-estimation/R/graph_construction.R` or a new `R/density_weighting.R` sourced by reports.

Deliverable: noisy-circle/tube report comparing base graph, density-weighted graph, Fermat, and oracle distances.

Phase 2: Robust Oracle Refinement

Extend `latent.tube.oracle()` with density caps, penalty caps, edge-density columns, and grid-resolution diagnostics.

Deliverable: updated latent-tube oracle benchmark.

Phase 3: Topology Before Weighting

Compare density-aware weighting on MST, CMST, iKNN, mKNN, repaired mKNN, and pruned iKNN graphs.

Deliverable: graph-topology sensitivity report.

Phase 4: Compositional Synthetic Data

Implement compositional cyclic gradients and test density spaces.

Deliverable: microbiome-like synthetic benchmark.

Phase 5: Public API Decision

Only after the above reports should density-aware helpers be considered for package-level API. The decision should be based on stability, diagnostics, biological interpretability, and downstream `gflow` improvement.

15. Open Design Questions

1. Should the primary graph-density summary be geometric mean or minimum?
2. Can edge-neighborhood density robustly detect shortcuts across gaps in high dimensions?
3. Should density floors be absolute, quantile-based, or estimator-specific?
4. Should default density normalization force median penalty to one?
5. How should intrinsic dimension be estimated for Fermat parameter correspondence?
6. Which density space is defensible for compositional count data?
7. Can random-walk local clouds become useful density diagnostics without confusing diffusion distance with geodesic distance?
8. What downstream `gflow` task should decide success: geodesic recovery, regression stability, gradient-flow interpretation, biological state separation, or all of these?

16. Summary Recommendation

Implement density-aware distance as an internal report-facing toolkit first. The first robust path is:

1. estimate or provide density;
2. transform density with explicit floors/caps;
3. summarize density on graph edges;
4. reweight an existing graph without changing topology;

5. compute graph shortest paths;
6. compare to Fermat/PWSPD, diffusion/PHATE, Euclidean, and oracle distances;
7. require density, edge, path, and tail diagnostics in every report.

Only after this workflow succeeds on noisy tubes, nonuniform curves, gaps, outliers, and compositional synthetic data should gflow expose a public density-aware geodesic API.